

Assignment - 2

Analysis of performance of 2 search algorithms:

- Linear Search
- Binary Search

Date Of Performance: 07/07/26

Date of Submission: 07/07/26

Name : Sakif Ahmed Rafi

Student ID : 00724105101024

Group : A1

Objective

Given a sorted array of size 10^5 . (sorted_random.txt). Compare the performance of two search algorithms -

i) Linear Search ii) Binary Search

And report the results found from analysis.

Tasks

1. Array Input

- Read the file sample.txt
- Use all the samples as input
- Store the numbers in an array

2. Algorithm Implementation

- Implement linear search, binary search

3. Performance

Measurement For
each run, measure:

Execution time (s)

Estimated energy consumption (J)

Energy (J) = [CPU's approximate average power](#) x Execution Time

(s) Peak memory usage (KB)

- Estimated CO₂ emissions

(Bangladesh) Energy (kWh) =

Energy (J) / 3,600,000 CO₂ (kg) =

Energy (kWh) x 0.62

Deliverables

- Source code implementing linear search, binary search.
- Filled results table and discussion.

Algorithm	Array Size	Execution Time (s)	Energy (J)	Peak Memory (KB)	CO ₂ Emissions (kg, BD)
Linear Search	10 ⁵	0.216506	14.0729	7760	2.42366e-06
Binary Search	10 ⁵	0.121699	7.91046	7936	1.36236e-06

Discussion / Conclusion

Linear Search: Took the highest execution time (0.216506 s) among the two algorithms. As a result, it also consumed more energy (14.0729 J) and produced higher estimated CO₂ emissions (2.42366×10^{-6} kg). Although it is simple to implement and works on any array, it is less efficient for searching in a large sorted dataset.

Binary Search: Achieved the lowest execution time (0.121699 s), making it the faster and more efficient algorithm. It also required less energy (7.91046 J) and generated lower estimated CO₂ emissions (1.36236×10^{-6} kg). The peak memory usage was almost the same as Linear Search, showing that Binary Search improves performance without significantly increasing memory usage. However, it requires the data to be sorted before searching.

Overall: Binary Search performed better than Linear Search in terms of execution time, energy consumption, and estimated CO₂ emissions for an array size of 10⁵. Both algorithms used nearly the same amount of memory, but Binary Search completed the search much more efficiently. Therefore, for large sorted datasets, Binary Search is the preferred choice, while Linear Search is more suitable for small or unsorted datasets.

Code for Performance Measurement

1.Binary Search :

```
#include <bits/stdc++.h>

#include <windows.h>

#include <psapi.h>

using namespace std;

#pragma comment(lib, "Psapi.lib")

using namespace std;

// Replace with your CPU's approximate average power (Watts)
constexpr double CPU_POWER_WATTS = 65.0;

// Bangladesh grid emission factor (kg CO2 / kWh)
constexpr double BD_EMISSION_FACTOR = 0.62;

int main() {

    LARGE_INTEGER freq, start, end;

    QueryPerformanceFrequency(&freq);

    QueryPerformanceCounter(&start);

    // ----- Your algorithm here -----
```

```

int n;

cin >> n;

vector<string> vec(n);

for(int i=0; i<n; i++)

    cin >> vec[i];

int t;

cin >> t;

string s;

while(t--){

    cin >> s;

    int l = 0, r = n-1, mid = 0;

    bool flag = false;

    while(l<=r){

        mid = (l+r)/2;

        if(s==vec[mid]){

            cout << s << " - found\n";

            flag = true;

            break;

        }

        else if(s<vec[mid]) r = mid-1;

        else if(s>vec[mid]) l = mid+1;

    }

    if(!flag){

        cout << s << " - not found\n";

    }

}

// -----

QueryPerformanceCounter(&end);

```

```

double elapsed = static_cast<double>(end.QuadPart - start.QuadPart) / freq.QuadPart;

double energy = CPU_POWER_WATTS * elapsed;    // Joules

double energy_kWh = energy / 3.6e6;          // kWh

double co2 = energy_kWh * BD_EMISSION_FACTOR; // kg CO2

// ----- Memory usage -----

PROCESS_MEMORY_COUNTERS_EX pmc;

if (GetProcessMemoryInfo(
    GetCurrentProcess(),
    reinterpret_cast<PROCESS_MEMORY_COUNTERS*>(&pmc),
    sizeof(pmc))) {

    SIZE_T peakMemUsed = pmc.PeakWorkingSetSize;

    cout << "Execution time: " << elapsed << " seconds\n";
    cout << "Estimated energy consumption: " << energy << " Joules\n";
    cout << "Peak Memory Usage: " << peakMemUsed / 1024 << " KB\n";
    cout << "Estimated CO2 emissions (Bangladesh): "
        << co2 << " kg\n";
}

else {
    cout << "Failed to get memory info.\n";
}

return 0;
}

```

2.Linear Search : (Only the linear search code part)

// ----- Your algorithm here -----

```
int n;

cin >> n;

vector<string> vec(n);

for(int i=0; i<n; i++)

    cin >> vec[i];

int t;

cin >> t;

string s;

while(t--){

    cin >> s;

    bool flag = false;

    for(int i=0; i<n; i++){

        if(vec[i]==s){

            cout << s << " - found" << endl;

            flag = true;

            break;

        }

    }

    if(!flag){

        cout << s << " - not found" << endl;

    }

}

// -----
```

